

# Polymarket US Combo RFQ Market-Maker

An event-driven service that receives combo requests for quote, computes a defensible fair value from live leg markets, applies inventory and uncertainty controls, and returns a two-sided price inside a 200 ms submission window.

## 200 ms

### Quote window

Target internal p99: 100 ms, leaving network and jitter headroom.

## 2-10

### Combo legs

Each leg carries a market symbol, combo side, and optional settlement value.

## 3 sec

### Last look

Reprice and re-run risk before confirming an accepted quote.

## RECOMMENDED STARTING POINT

Build a single-region, stream-first service with an in-memory hot path and an append-only audit log. Start in observe-only mode, then shadow quoting, then live trading with tiny limits. Do not add correlation or automatic hedging until the independent-leg baseline is measured against real RFQs.

## TARGET

**Polymarket US Exchange API.** Combo RFQ uses gRPC/protobuf. The international REST combo endpoint is not part of this design and is unsuitable for a US-based operator.

## SAFETY POSTURE

**Fail closed.** Stale books, missing legs, uncertain state, breached limits, clock drift, or unhealthy Drop Copy all prevent confirmation.

# Contents and decision summary

|    |                                      |    |
|----|--------------------------------------|----|
| 01 | Purpose, scope, and success criteria | 3  |
| 02 | Polymarket US contract               | 4  |
| 03 | System architecture                  | 5  |
| 04 | RFQ lifecycle and recovery           | 6  |
| 05 | Pricing engine                       | 7  |
| 06 | Quote construction and latency       | 8  |
| 07 | Risk, last look, and hedging         | 9  |
| 08 | State, observability, and recovery   | 10 |
| 09 | Technology and security              | 11 |
| 10 | Testing and rollout                  | 12 |
| 11 | Open questions and sources           | 13 |

## KEY DESIGN DECISIONS

- **Stream-first:** one RFQ event stream; durable reads only at startup, reconnect, and targeted reconciliation.
- **Hot path stays local:** no database calls or external model calls while pricing an RFQ.
- **Two-stage pricing:** independent-leg model first; correlation model only after baseline telemetry exists.
- **Separate decision from execution:** pricing proposes; risk authorizes; quote manager transmits.
- **Drop Copy owns fills:** RFQ events describe workflow, not canonical execution.
- **Shadow by default:** calculate and log; never call `confirmQuote` until live mode is explicitly enabled.
- **Language remains open:** Python and TypeScript are both viable; choose after a latency and operations spike.

## PRE-PRODUCTION GATE

Confirm combo maker enrollment, credentials and roles, production rate limits, fee and rebate treatment, and approved market-data channels with Polymarket US before sending a live quote.

# Quote quickly, but never outrun the risk state

The service's job is narrow: observe every eligible combo RFQ, derive a fair probability from current component markets, return competitive bid and ask prices, and safely confirm selected quotes. It is not a general sports model, a retail trading UI, or a portfolio optimizer.

## FUNCTIONAL REQUIREMENTS

- Consume public `rfq_created` events and recover open RFQs after reconnect.
- Maintain fresh best bid/offer and depth for every tradable leg.
- Compute both requester-buy and requester-sell economics.
- Submit or replace at most one deterministic quote per maker and RFQ.
- Handle private acceptance, confirmation, execution, and deletion events.
- Reconcile fills through Drop Copy and update exposure.
- Generate hedge intents under independent limits.

## NON-FUNCTIONAL REQUIREMENTS

- **Latency:** p99 internal decision under 40 ms; end-to-end quote acknowledgement target under 100 ms.
- **Availability:** reconnect automatically, then restore state from durable reads before quoting.
- **Correctness:** deterministic rounding, idempotent event handling, explicit state transitions.
- **Auditability:** reproduce any quote from market snapshot, model version, limits, and configuration.
- **Containment:** any dependency failure disables quoting or confirmation without stopping recovery.

## SUCCESS MEASURES FOR THE PILOT

| MEASURE           | TARGET         | WHY IT MATTERS                   |
|-------------------|----------------|----------------------------------|
| RFQ coverage      | > 99% observed | Proves stream and recovery       |
| Decision latency  | p99 < 40 ms    | Preserves network budget         |
| Stale-book quotes | 0 confirmed    | Primary adverse-selection guard  |
| State mismatches  | 0 unresolved   | Durable recovery must converge   |
| Reproducibility   | 100% sampled   | Explains every price and decline |
| Risk breaches     | 0              | Limits are hard, not advisory    |

## OPERATING PRINCIPLE

Being first wins exact price ties, but a fast bad quote is worse than no quote. The bot quotes only when all inputs are present, recent, and internally consistent.

# The US combo workflow is gRPC, stateful, and deadline-driven

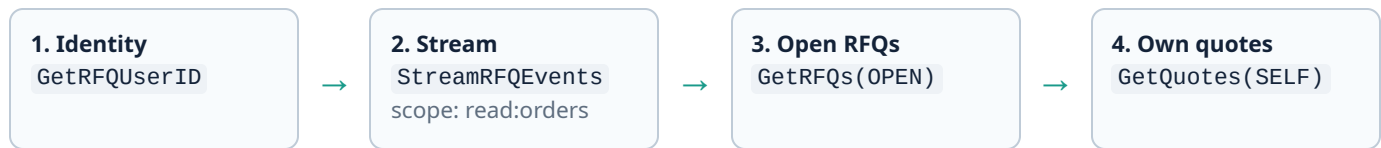
## SERVICES AND SCHEMAS

- `polymarket.v1.ComboAPI`: `CreateCombo`, `GetCombos`.
- `polymarket.v1.RFQAPI`: RFQ and quote reads/writes plus `StreamRFQEvents`.
- Generate clients from `combo.proto` and `rfq.proto`. The retired `combos.proto` is out of contract.
- Unary calls may use REST or gRPC; RFQ events are gRPC-only. This design uses gRPC consistently.

## RFQ PAYLOAD

- Combo symbol and ordered `comboLegs`.
- Either exact `qtyDecimal` or `cashOrderQty`.
- Pseudonymous requester ID, rest-remainder flag, and open/closed state.
- Each leg includes symbol, combo side, and optional raw YES/LONG `settlementPrice`.

## MAKER STARTUP SEQUENCE



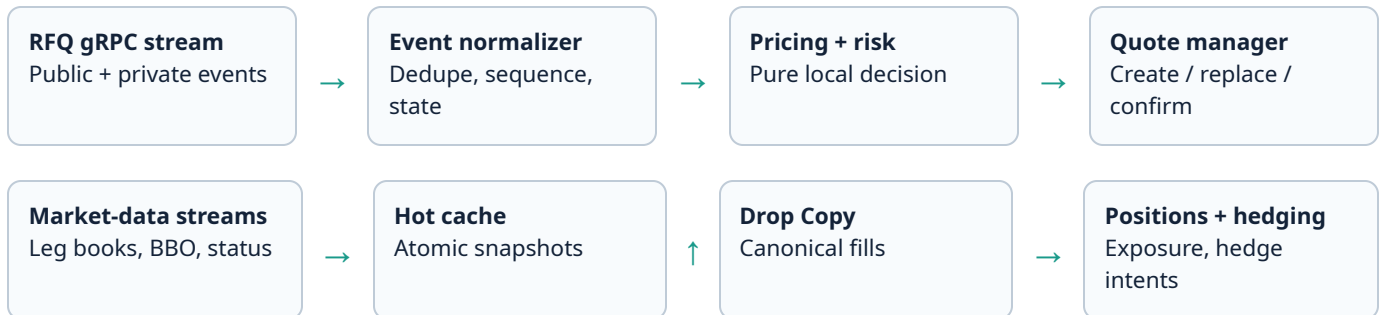
## QUOTE SEMANTICS

| FIELD                  | ECONOMIC MEANING           | SELECTION             |
|------------------------|----------------------------|-----------------------|
| <code>buyPrice</code>  | Maker ask; requester buys  | Lowest positive wins  |
| <code>sellPrice</code> | Maker bid; requester sells | Highest positive wins |
| <code>"0"</code>       | Side unavailable           | Ignored               |

A quote must include `account` and `restRemainder`; `postOnly` is optional. Do not send side, symbol, quantity, expiration, or client request ID: the RFQ supplies them. Repeating `CreateQuote` replaces economics under the same deterministic quote ID.

**Stream contract:** events are best-effort with no replay, ordering, or deduplication guarantee. On every reconnect, pause quoting and repeat durable reads before resuming.

## Keep the 200 ms path in memory and move everything else off-path



### LATENCY-CRITICAL PROCESS

- **RFQ stream consumer:** parses events, detects duplicates, and dispatches work without blocking the stream reader.
- **Market-data cache:** stores immutable per-symbol snapshots with exchange time, receive time, sequence, BBO, and depth.
- **Pricing engine:** transforms leg marks into combo fair value and uncertainty.
- **Risk module:** enforces eligibility, staleness, inventory, size, price-band, and loss controls.
- **Quote manager:** rounds to tick, sends one dual-sided quote, replaces only if economics move enough, and handles last look.

### ASYNCHRONOUS CONTROL PLANE

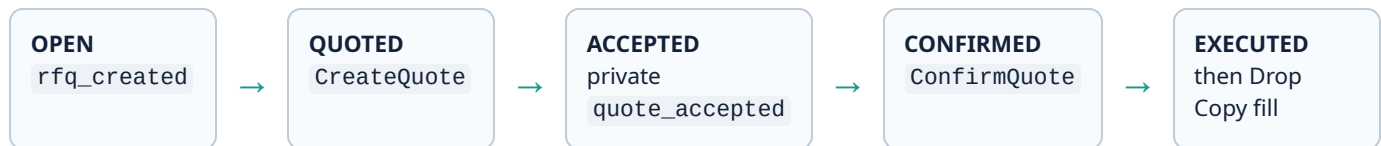
- **Drop Copy consumer:** reconciles fills and order IDs; updates the canonical ledger.
- **Position service:** computes combo and leg-equivalent exposure.
- **Hedging engine:** emits bounded orders; never shares a failure domain with quoting.
- **Event store:** persists raw messages, decisions, configurations, and model versions.
- **Operations API:** health, metrics, kill switch, mode changes, and read-only diagnostics.

### DEPLOYMENT SHAPE

Run one active quoter per account and region. A warm standby may consume market data and reconstruct state, but it must not quote until it holds an exclusive lease. Keep the database and telemetry exporters off the critical thread. If persistence is unavailable, buffer locally for a bounded period; if the buffer fills, stop quoting.

**Recommended boundary:** one service binary for the pilot, internally modularized. Split into independent services only after profiling shows a scaling or isolation need; network hops inside the hot path are avoidable latency.

# RFQ events trigger work; durable state decides what is true



| EVENT OR STATE  | REQUIRED BOT ACTION                       | HARD GUARD                  |
|-----------------|-------------------------------------------|-----------------------------|
| rfq_created     | Price immediately; quote or log decline   | All legs fresh and tradable |
| rfq_closed      | Stop creates/replaces; retain quote state | Do not infer selection      |
| quote_accepted  | Run last look; confirm or delete          | Use authoritative deadline  |
| quote_confirmed | Expect paired submission                  | Track execution deadline    |
| quote_executed  | Record embedded durable quote state       | Still wait for Drop Copy    |
| Drop Copy fill  | Book position and launch hedge policy     | Idempotent execution ID     |

## RECONNECT ALGORITHM

1. Set runtime state to `RECOVERING`; disable create, replace, and confirm.
2. Reopen the RFQ stream.
3. Read all open RFQs and the maker's quotes.
4. Reconcile by RFQ ID and deterministic quote ID.
5. Refresh positions and Drop Copy cursor.
6. Resume only after invariants pass.

## UNKNOWN WRITE RESULT

If a write times out or the connection fails, never retry blindly. Query the exact RFQ or quote using `GetQuotes`, decide whether the intended state already exists, then issue an idempotent replacement or stop.

Opaque cursors are scoped to the same participant, query path, and filters. Store those dimensions beside every cursor.

**Never confirm on stream state alone.** At last look, compare the accepted quote against the latest market snapshot, current limits, and the durable quote's `confirmationDeadline`.

# Start with transparent leg probabilities, then earn edge through correlation

## LEG MARK CONSTRUCTION

For each leg, compute a probability mark from its live book. Prefer a bounded microprice using best bid, best ask, and displayed size; fall back to midpoint only when both sides are present. Reject crossed books, halted markets, missing sides, sequence gaps, or data older than the configured threshold.

```
mid_i      = (best_bid_i + best_ask_i) / 2
micro_i    = (ask_size_i * best_bid_i + bid_size_i * best_ask_i)
            / (bid_size_i + ask_size_i)
p_i        = clamp(micro_i, best_bid_i, best_ask_i)
q_i        = p_i for a BUY/YES leg; 1 - p_i for a SELL/NO leg
```

A present leg `settlementPrice` is the raw YES/LONG result in [0,1]. Preserve it as received, then apply side transformation inside pricing; do not mutate the source field.

### V1: INDEPENDENT LEGS

For unresolved legs, the combo fair probability is the product of favorable leg probabilities:

```
fair_independent = product(q_i)
```

Resolved winning legs contribute 1; resolved losing legs make the combo worth 0. This model is deterministic, explainable, and fast enough for the hot path.

### UNCERTAINTY ESTIMATE

Attach an uncertainty term rather than treating the point estimate as truth. Increase it with:

- Wide or shallow leg books
- More legs
- Stale or recently gapped streams
- Same-game or logically linked legs
- Large RFQ size versus visible depth

## V2: CORRELATION MODEL

There is no official combo pricing formula. Same-game correlations are therefore both the maker's potential edge and largest model risk. Add correlation only after V1 creates a clean dataset of RFQs, leg marks, outcomes, and counterfactual profit and loss.

- **First implementation:** group legs by event and apply a calibrated log-odds adjustment by relationship type, such as favorite plus spread or team win plus game total.
- **Later implementation:** a Gaussian-copula or simulation model using event-level latent factors, constrained to preserve each leg's marginal probability.
- **Guardrail:** correlation adjustments are bounded; unknown relationships widen the spread or suppress quoting instead of guessing.

**Model calibration rule:** use only data available at quote time. Store the exact leg snapshot and model version to prevent look-ahead bias in backtests.

# Turn fair value into a valid, competitive, inventory-aware quote

```
half_spread = base_edge
              + model_uncertainty
              + depth_impact(size)
              + event_risk
              + operational_buffer

center       = fair_value - inventory_skew
buyPrice     = round_up_to_tick(center + half_spread) # maker ask
sellPrice    = round_down_to_tick(center - half_spread) # maker bid
```

## VALIDATION BEFORE SUBMIT

- Clamp each positive price to instrument limits.
- Round outward to the current combo `tickSize`; current instruments use 0.001.
- For quantity RFQs, apply `qtyDecimal` to each offered side.
- For cash RFQs, check the API-derived quantity `floor(cashOrderQty / sidePrice)` against minimum size.
- After creation, store persisted `buyQtyDecimal` and `sellQtyDecimal`; do not recompute them.
- Set a side to "0" when risk or sizing makes it unavailable; at least one side must remain positive.

## REPLACEMENT POLICY

Because each maker has one deterministic quote ID per RFQ, replacement is a state update. Replace only when:

- Fair value moves by at least one tick plus a configurable hysteresis;
- Inventory skew changes materially; or
- A side becomes ineligible.

Stop all creates and replacements when `rfq_closed` arrives, while retaining state for a later private acceptance event.

## 200 MS LATENCY BUDGET

| STAGE             | BUDGET        | IMPLEMENTATION RULE            |
|-------------------|---------------|--------------------------------|
| Decode + dispatch | 3 ms          | Non-blocking stream reader     |
| Snapshot lookup   | 5 ms          | Atomic in-memory reads         |
| Pricing           | 20 ms         | No network or database         |
| Risk checks       | 7 ms          | Precomputed exposure totals    |
| Build request     | 5 ms          | Generated protobuf client      |
| Network + ack     | 60 ms         | Warm channel, colocated region |
| <b>Reserve</b>    | <b>100 ms</b> | Jitter and exchange handling   |

**Deadline policy:** the 200 ms window, three-second last look, and one-second execution delay are exchange configuration. Use server-provided deadlines from durable state, never locally reconstructed timers.

# Last look is a fresh trade decision, not a ceremonial confirm

## PRE-QUOTE GATES

- Mode permits quoting.
- RFQ is open and not previously terminal.
- Every unresolved leg is mapped and tradable.
- Book age and stream sequence are within limits.
- Per-quote notional and projected position pass.
- Both price and derived size are valid.
- Loss, reject-rate, latency, and operational kill switches are clear.

## LAST-LOOK GATES

- Authoritative `confirmationDeadline` leaves a safety buffer.
- Accepted side and persisted quantity match local state.
- Current fair value remains inside max slippage.
- Fresh projected exposure still passes every limit.
- Drop Copy, market data, clock sync, and trading channel are healthy.
- Otherwise call `DeleteQuote` before the deadline.

## MINIMUM LIMIT SET

| LIMIT                      | SCOPE               | ACTION ON BREACH    |
|----------------------------|---------------------|---------------------|
| Per-quote notional         | RFQ + accepted side | Suppress side       |
| Per-combo inventory        | Combo symbol        | Skew or suppress    |
| Event exposure             | Shared game/event   | Widen or stop       |
| Gross and net inventory    | Trading account     | Stop confirmation   |
| Daily realized + mark loss | Service/account     | Global kill switch  |
| Adverse-selection score    | Model/segment       | Widen, then disable |
| Stale data / latency       | Dependency          | Fail closed         |

## HEDGING POLICY

A combo is a digital conjunction, so its risk is not perfectly replicated by independently trading its legs. Estimate leg deltas as the change in combo value for a small change in each leg probability, then generate bounded hedge intents against available leg liquidity. Track residual correlation and basis risk explicitly.

- **Phase 3:** manual or advisory hedging; tiny inventory limits.
- **Phase 4:** automatic hedging only after Drop Copy-to-position reconciliation is proven.
- **Isolation:** a hedge failure raises risk and may stop new quotes, but cannot corrupt the quote engine's state.

**Kill switch behavior:** stop creates, replacements, and confirms; delete active accepted quotes when possible; cancel resting hedge orders; continue consuming streams and reconciling fills.

# An append-only decision record makes a best-effort stream operable

## STATE MODEL

| STORE           | CORE FIELDS                                | RETENTION PURPOSE  |
|-----------------|--------------------------------------------|--------------------|
| rfq_events      | event ID/type, RFQ ID, raw proto, times    | Replay and dedupe  |
| rfqs            | symbol, legs, size mode, status            | Durable projection |
| quote_decisions | fair, spread, bid/ask, decline reason      | Audit and backtest |
| quotes          | quote ID, prices, persisted qty, deadlines | Recovery           |
| executions      | exchange IDs, side, price, quantity        | Canonical ledger   |
| positions       | combo, event, gross/net, marks             | Real-time risk     |
| risk_events     | limit, observed, threshold, action         | Control evidence   |
| configs         | version, hash, approver, activation        | Reproducibility    |

## IDEMPOTENCY INVARIANTS

- Unique RFQ event key when provided; otherwise stable hash of event type, RFQ ID, quote ID, and exchange timestamp.
- One projected RFQ row per RFQ ID.
- One quote row per deterministic quote ID.
- One execution row per Drop Copy execution ID.
- State transitions are monotonic except a documented quote replacement back to active.

## OPERATIONAL METRICS

- RFQs seen, eligible, quoted, accepted, confirmed, executed.
- End-to-end and stage latency percentiles.
- Quote win rate, last-look decline rate, and exchange rejects.
- Markout at 1 s, 10 s, close, and settlement.
- Spread captured, realized P&L, model P&L, hedge cost.
- Stream disconnects, gaps, reconciliation deltas, stale-book suppressions.

## ALERTS AND RUNBOOKS

Page immediately for kill-switch activation, unresolved position mismatch, Drop Copy lag, stale market data, credential failure, repeated quote rejects, or clock drift. Lower-severity alerts cover rising markout losses, quote latency erosion, and model-segment underperformance.

**Rate-limit posture:** use streams for live state. Reserve `GetRFQs` and `GetQuotes` for startup, reconnect, unknown write results, and targeted reconciliation. On a 429, stop requests and back off; confirm firm-specific production limits before launch.

## Choose the language by team fit; keep the protocol and controls identical

| CRITERION          | PYTHON                                  | TYPESCRIPT / NODE.JS            |
|--------------------|-----------------------------------------|---------------------------------|
| Pricing research   | Strong numerical ecosystem              | Good; fewer quant defaults      |
| gRPC / protobuf    | Mature code generation                  | Mature code generation          |
| Concurrency        | <code>asyncio</code> ; isolate CPU work | Natural event-loop model        |
| Shared web tooling | Separate frontend stack                 | Strong code sharing             |
| Latency control    | Good with bounded hot path              | Good with event-loop discipline |
| Best fit           | Model-heavy small team                  | Service-heavy TS team           |

**Decision method:** implement the same replay benchmark in both languages: generated gRPC client, 10-leg snapshot lookup, V1 pricing, risk checks, protobuf serialization, and 10,000 recorded RFQs. Choose on measured p99 latency, profiling clarity, deployment reliability, and who will maintain it.

### SUGGESTED PLATFORM

- Generated protobuf clients pinned to a reviewed schema version.
- Single event loop plus bounded worker pool if needed.
- In-process hot cache; PostgreSQL for durable events and projections.
- Prometheus-compatible metrics, structured logs, distributed traces.
- Containerized deployment in the lowest-latency supported US region.

### CONFIGURATION DISCIPLINE

- Version every model, limit set, feature flag, and symbol mapping.
- Separate development, paper, and production accounts.
- Require explicit promotion for live mode and limit increases.
- Validate configuration at startup; unknown fields are fatal.
- Store activation events with actor and hash.

### AUTHENTICATION AND SECRET HANDLING

- The Exchange API is intended for onboarded participant firms and uses Auth0/RSA authentication. Confirm the exact credential and token flow during onboarding.
- Keep private keys and client secrets in a managed secret store; inject short-lived tokens at runtime. Never write credentials to source, images, logs, crash dumps, or replay fixtures.
- Use least-privilege principals: RFQ trading, market-data reads, Drop Copy, and operations access should be separated where the platform permits.
- Restrict outbound network destinations, rotate secrets, audit access, and synchronize clocks with monitored NTP. Deadline logic depends on accurate time.
- The operations endpoint must require strong authentication and authorization; the kill switch is privileged but must remain fast and available.

# Prove recovery and pricing before the bot is allowed to trade

## TEST LAYERS

1. **Unit:** side transforms, settlement handling, cash sizing, tick rounding, clamps, inventory skew, deadlines.
2. **Property:** fair value stays in  $[0,1]$ ; asks never fall below bids; wider uncertainty never tightens spread; resolved loser prices at zero.
3. **Protocol contract:** generated clients against a test server implementing current proto schemas.
4. **Replay:** recorded RFQs and books through deterministic clocks; byte-for-byte decision comparison.
5. **Fault injection:** duplicate/out-of-order events, reconnects, missing legs, stale books, timeouts, unknown writes, late Drop Copy.
6. **Load:** burst RFQs plus market updates; verify p99 latency and no stream backpressure.

## SAFETY MODES

**OBSERVE:** consume, reconcile, and record. No prices generated.

**SHADOW:** calculate counterfactual quotes and markouts. No `CreateQuote` or `ConfirmQuote`.

**UAT:** submit only in an exchange-approved test environment. Confirmation disabled unless the test case requires it.

**LIVE\_TINY:** create and confirm under minimal notional, inventory, and daily-loss limits.

**LIVE:** production limits; correlation and auto-hedging gated separately.

The runtime starts in the safest configured mode. A restart must never silently escalate privileges or trading mode.

## PROMOTION EVIDENCE

| GATE                 | EVIDENCE REQUIRED                                           |
|----------------------|-------------------------------------------------------------|
| Observe to shadow    | Stable stream; reconnect converges; complete leg maps       |
| Shadow to UAT        | Latency target; zero invalid quotes; replay deterministic   |
| UAT to live tiny     | Last-look and Drop Copy end-to-end; kill switch tested      |
| Live tiny to scale   | Positive net markout after fees/hedges; no control breaches |
| V1 to correlation    | Out-of-sample lift; bounded error by relationship type      |
| Manual to auto-hedge | Position reconciliation and hedge simulator proven          |

**Paper-trading nuance:** repeatedly sending attractive live quotes and then declining every acceptance can harm counterparties and may violate maker expectations. Use local shadow quotes unless Polymarket explicitly approves a no-confirmation test mode.

## Scale only after each layer produces evidence

| PHASE               | BUILD                                                                        | EXIT CONDITION                                               |
|---------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| 1. <b>Read-only</b> | Auth, proto clients, RFQ stream, durable recovery, leg-data cache, event log | Stable replay and reconnect; complete audit trail            |
| 2. <b>Shadow</b>    | V1 pricing, quote validator, risk rules, counterfactual markouts, dashboards | Latency and validity targets; acceptable simulated economics |
| 3. <b>Live tiny</b> | Create/replace, last look, Drop Copy, positions, kill switch, manual hedges  | No reconciliation gaps; controlled realized outcomes         |
| 4. <b>Scale</b>     | Correlation V2, adaptive spreads, automated hedging, refined inventory model | Out-of-sample model lift and stable risk-adjusted P&L        |

### OPEN EXTERNAL QUESTIONS

1. **Maker access:** Is combo RFQ quoting open to all eligible Exchange API participants, or separately enrolled/whitelisted?
2. **Production rate limits:** What per-firm gRPC limits apply to RFQ reads, creates, replaces, confirms, and deletes?
3. **Fees and rebates:** What US fee schedule, maker incentives, and reporting treatment apply to combos and leg hedges?
4. **FIX roadmap:** What is the supported timeline and feature parity for combo RFQ over FIX?
5. **Test environment:** Is there a sandbox with synthetic RFQs, Drop Copy, and deadline behavior?

### INTERNAL DECISIONS BEFORE CODING

1. Choose Python or TypeScript after the benchmark spike.
2. Define eligible sports, maximum legs, and excluded correlation patterns.
3. Approve starting notionals, inventory, loss, and staleness limits.
4. Select market-data depth and leg-mark algorithm.
5. Define who can enable live mode, raise limits, and trigger/clear the kill switch.
6. Set retention and review policy for RFQ, quote, and execution records.

### FIRST IMPLEMENTATION SLICE

Deliver Phase 1 plus the V1 pricing library behind a shadow-only mode. The executable demo should reconnect cleanly, replay a captured session deterministically, price a 10-leg RFQ without I/O on the hot path, and explain every quote or decline with a structured reason code.

### SOURCES

1. [Polymarket US, "Combos" trader guide](#) - RFQ services, startup, quote semantics, selection, deadlines, recovery, and Drop Copy guidance. Accessed 16 September 2026.
2. [Polymarket US, "API Overview"](#) - Exchange versus Retail APIs, protocols, authentication model, and participant/account concepts. Accessed 16 September 2026.
3. [Polymarket US, "Rate Limits"](#) - stream-first guidance, caching, 429 behavior, and automated-system escalation path. Accessed 16 September 2026.

**Scope note:** This document describes software architecture and controls, not a guarantee of profitability or authorization to trade. Production use depends on Polymarket US onboarding, applicable rules, and approved account permissions.